CosmoQ Cinema: Web Ticket Booking and Cinema Management with Flutter and ASP.NET Core

Nguyen Nguyen Ngu (QE190064), Dao Thi Thanh Huong (QE190046), Pham Ngo Bich Tram (QE1900100)
Nguyen Hoang Ngoc Chau (QE190088), Nguyen Xuan Dang Khoa (QE1900126)
Student Developers, Class: SEK19B.Net
Department of Software Engineering
FPT University Quy Nhon Campus, Binh Dinh, Vietnam
Huynh Vang — Project Advisor, Lecturer
Department of Software Engineering, FPT University Quy Nhon Campus
Binh Dinh, Vietnam | vangh@fe.edu.vn

***Abstract*—Cinema ticketing requires customer purchases and cinema operations to agree on showtimes, seat ownership, and payment state. CosmoQ Cinema implements these interactions through Flutter Web customer and administration entry points and a shared ASP.NET Core 8 backend. This report reconstructs the implementation from source, dependency manifests, SQL schemas, tests, and deployment scripts. A YARP gateway routes JSON requests to authentication, movie, and booking services. Entity Framework Core with Npgsql accesses six PostgreSQL databases; transactional reservation rows and a seat/showtime uniqueness constraint support seat contention handling. The browser layer uses named routes, explicit session state, remote repositories, responsive layouts, and token-refresh coordination. PayOS checkout and reconciliation, RabbitMQ outboxes, notification records, counter sales, promotions, and loyalty functions are analyzed within their verified boundaries. Seventeen isolated booking-service unit tests and twenty-one gateway tests passed; backend compilation succeeded, with four nullable warnings in MovieService. Container-dependent verification was blocked by an unavailable Docker daemon. The Flutter suite passed 209 tests; both browser entry points compiled, and analysis retained one informational lint. These checks remain separate from source implementation, and no live payment or production deployment is claimed. The contribution is a traceable account of the Web system's coordination mechanisms and remaining verification obligations.**

***Index Terms*—Cinema booking, Flutter Web, ASP.NET Core, PostgreSQL, JWT, PayOS, RabbitMQ, cinema management.**

I. Introduction

Cinema ticketing couples a customer-facing purchase with operational tasks such as maintaining schedules, allocating seats, collecting payments, and admitting customers. A booking interface must therefore coordinate information that can change between browsing a show and completing checkout. A seat visible as free is not necessarily still available when a customer submits an order. Likewise, a browser returning from a payment page is not sufficient evidence that the cinema has received money. These are the concrete coordination problems addressed by the inspected CosmoQ implementation, rather than measured claims about the shortcomings of a particular cinema's previous process.

CosmoQ Cinema implements customer booking and cinema-management workflows through Flutter browser applications and an ASP.NET Core backend. This paper examines the Web system: the customer application built from `main.dart` and the administration application built from `main_admin.dart`. They are distinct entry points in one Flutter source project, not independently verified frontend repositories. A mobile build shares customer code and backend APIs, but its platform behavior is outside the primary scope of this paper. Repository evidence, including source-level dependencies and SQL, establishes the implementation described here [1]–[4].

The principal engineering contributions are the following integrated mechanisms:

- **Browser workflow composition.** Named navigation, session recovery, responsive customer pages, and role-aware management entry points share remote API adapters.
- **Server-controlled reservation.** Backend pricing, transactional seat holds, and a seat/showtime uniqueness constraint coordinate competing purchases.
- **Payment and operational follow-up.** Provider reconciliation and outbox events connect booking state to notifications, refunds, and cinema operations.

Concessions, promotions, loyalty balances, reviews, check-in, and notification records support these workflows. Their source implementation does not establish a production deployment, successful live payment, or performance improvement over another ticketing system.

A. Evidence and Scope

Inspection used repository revision `55642353331f` and database submodule revision `e455d297c76b`, retrieved on October 28, 2026. Manifests, entry points, controllers, services, repositories, entity mappings, SQL, platform configuration, tests, and CI definitions were cross-checked. Existing requirements documents served as navigation aids rather than proof that a feature works. Official framework references explain the technologies, while repository references substantiate CosmoQ-specific behavior.

The analysis distinguishes implemented source, mocked test dependencies, configured external integrations, and behavior

TABLE I
VERIFIED WEB TECHNOLOGY STACK

Component	Technology	Responsibility
Browser UI	Flutter, Dart, Material	Customer and administration views
Client API	Dart `http`	JSON requests and bearer tokens
Gateway	ASP.NET Core 8, YARP 2.3.0	Routing and request policies
Backend	ASP.NET Core 8	Controllers and business services
Persistence	EF Core 8, Npgsql; PostgreSQL 17 image	Mappings and SQL databases
Events	RabbitMQ.Client 7.2.1	Outbox publication and consumers
Payment	PayOS HTTP integration	Checkout links and reconciliation
Operations	Docker Compose; OpenTelemetry	Local topology and telemetry

not executed during this review. In particular, a test using a fake PayOS client cannot validate a merchant account, banking settlement, or public webhook delivery. Section VII reports the checks actually executed and their limits.

II. SYSTEM ARCHITECTURE OVERVIEW

A. Deployment Units and Communication

Figure 1 summarizes the verified Web topology. Browser requests use JSON over HTTP through a YARP gateway. AuthService manages identity; MovieService contains movie and showtime/room functionality; BookingService contains booking, payment, notification, and related business services. Each is an ASP.NET Core executable. Their separate Compose service definitions and inter-service HTTP/event communication support describing the backend as a service-based deployment. This does not imply independently scalable production infrastructure or a separate service for every business module [3], [5]–[7].

The gateway listens on host port 5000 in Compose. Backend and infrastructure debug ports are generally bound to loopback, while the gateway is accessible from the network. Route-to-cluster configuration selects the destination service; downstream failures are translated into structured 503 or 504 responses. BookingService calls MovieService for authoritative showtime and seat data, forwarding the acting user's bearer token. RabbitMQ carries business events after local transactions commit. Browser clients do not connect directly to PostgreSQL or RabbitMQ [3], [5], [7].

B. Technology and Repository Organization

Table I identifies technologies confirmed by dependencies and active code. The installed verification SDK is Flutter 3.47.2 with Dart 3.13.2; the Flutter CI workflow instead requests version 3.47.4. This is a reproducibility distinction, not evidence that either version was used for a released website. Backend projects target `net8.0`; the local build host uses .NET SDK 9.0.305.

The Flutter `lib` tree separates `app`, `admin`, `core`, `data`, and feature-oriented directories. `AppDependencies.remote` constructs real service adapters, repositories, secure token storage, and a token manager. Backend services register controller, business-service, repository, and EF Core dependencies explicitly. Interface-backed repositories are present, but a uniform Clean Architecture or domain-driven design is not established by this organization. Flutter's architecture guidance provides context for separation of UI and data access [8]; CosmoQ's actual dependency construction remains the relevant evidence [1].

C. Persistence Boundaries

Compose initializes six databases on one PostgreSQL instance. Their domain suffixes are `auth`, `movie`, `showtime`, `booking`, `payment`, and `notification`; full names follow the `cinema_..._db` convention. MovieService owns two EF contexts and BookingService owns three. Thus six database names do not imply six backend executables. Npgsql supplies the EF Core provider [9]; source registration selects each database [3], [6], [7], [10].

III. BACKEND DESIGN AND IMPLEMENTATION

A. Identity and Access Services

Account lifecycle. AuthService implements registration, email verification, login, refresh, logout, profile changes, password changes, and administrative account operations. Passwords and OTP values are hashed with BCrypt. JWT generation uses HMAC-SHA256 and includes user identity and role claims. Refresh tokens are generated from 64 cryptographically random bytes, stored in the authentication database, and revoked when rotated. A logout revokes the supplied refresh token; it is not described here as an instantaneous blacklist of every previously issued access token [10].

Request authorization. JWT validation checks issuer, audience, signature, and lifetime. Gateway policies and downstream controller policies restrict privileged actions. Protected gateway requests also query AuthService's current-account endpoint. Disabled accounts are refused, and a mismatch between current roles and token roles returns a role-change error. Failure to perform that check returns 503 rather than forwarding an unchecked protected request [5]. These are concrete bearer-token mechanisms consistent with the ASP.NET Core model [11], not a claim of OAuth/OpenID Connect federation.

B. Catalog and Schedule Support

MovieService provides movie browsing, detail retrieval, genres, title search, and administrative editing. Its showtime code validates room membership of requested seats, rejects shows that are not open or have started, and enforces paired sweetbox selection when a partner exists. Seat prices derive from the showtime price, presentation-format surcharge, and seat-type surcharge. Room, seat, and schedule APIs support

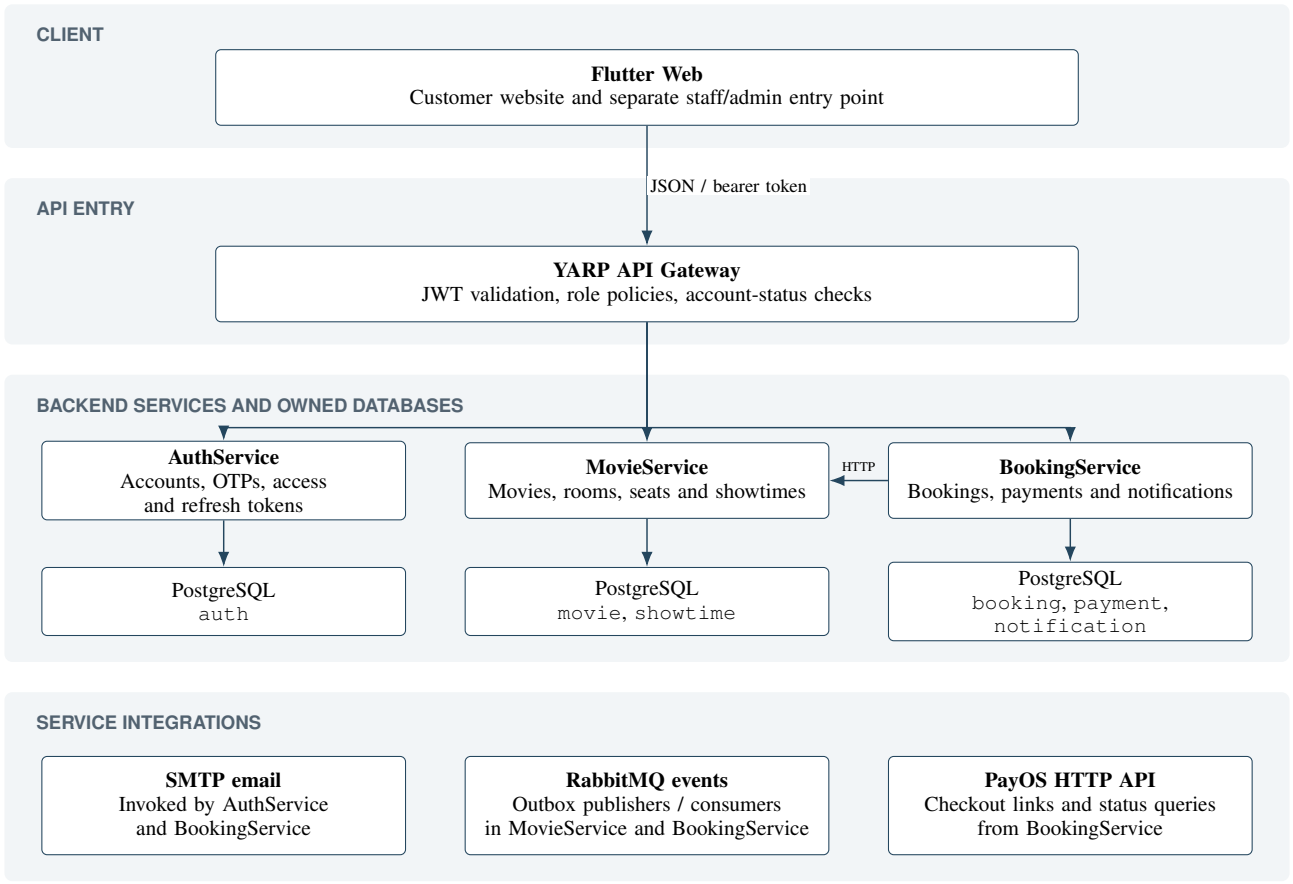


Fig. 1. Web system architecture with explicit service/database ownership and external integration bindings. Solid arrows show client/API and persistence paths. The integration cards identify the backend services responsible for each external binding. Database labels abbreviate the six `cinema_..._db` names; all six use one PostgreSQL instance in Compose.

management screens rather than requiring the browser to manipulate database records [2], [6].

Showtime creation checks start/end ordering and room schedule conflicts through the repository. Cancellation records a business event in the showtime database outbox; BookingService consumes it to process affected bookings. These checks show implemented business validation. They do not establish that all possible concurrent administrative schedule edits have been tested in this environment.

C. Booking Transactions and Seat Contention

The booking controller derives the customer ID and email from authenticated claims for self-service purchases. The service rejects duplicate seat IDs, requests server-side seat prices, and calculates concessions from the active menu. It loads showtime timing for later cancellation policy decisions. The movie title normally comes from MovieService, although the code retains a request-title fallback if no title is returned; therefore it would be inaccurate to claim that every display field is exclusively server-derived [7].

The transaction uses three complementary mechanisms [4], [7], [12]:

- **Lock existing reservations.** PostgreSQL `FOR UPDATE` locks the selected rows; booked seats and unexpired holds are rejected.
- **Constrain new reservations.** The unique `uq_seat_per_showtime` key protects a seat/showtime pair before a row exists to lock. A unique violation rolls back the transaction and returns a conflict.
- **Commit related state.** The booking, seat snapshots, holds, and outbox event are persisted in one local transaction.

These are specific concurrency mechanisms; they do not establish an unrestricted guarantee under every failure mode.

Listing 1 reproduces the relevant EF mapping. The database schema contains the corresponding unique constraint. UI disabling alone would not provide this protection; simultaneous browser sessions can submit the same selection despite a recently refreshed map.

```
entity.HasIndex(
    e => new { e.ShowtimeId, e.SeatId },
    "uq_seat_per_showtime")
  .IsUnique();
```

Listing 1. Reservation uniqueness mapping from BookingDbContext.

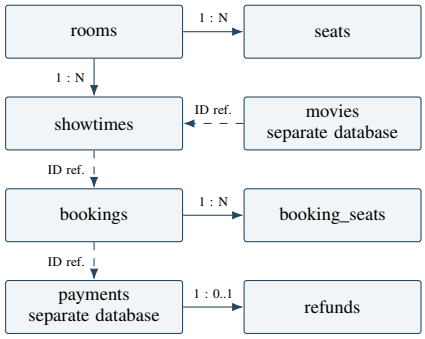


Fig. 2. Selected schema relationships reconstructed from SQL. Solid arrows denote parent-to-child SQL foreign keys; dashed arrows denote logical ID references. Reservations, identity, and other extension tables are omitted.

TABLE II
REPRESENTATIVE BROWSER SCREENS AND ROUTES

View	Route or entry	Responsibility
Customer home	`/home`	Browse and filter movies
Movie details	`/movies/{id}`	Metadata, reviews, showtimes
Seats	`/showtimes/{id}/seats`	Selection and availability
Booking detail	`/bookings/{id}`	Ticket and booking state
Payment	`/bookings/{id}/payment`	Checkout and reconciliation
Movie management	`/admin/movies`	Admin catalog editing
User management	`/admin/users`	Account and role operations
Staff workspace	`main_admin.dart`	Counter sales and operations

D. Database Model and Relationships

Figure 2 shows a selected, verified relational subset. Solid edges represent declared SQL foreign keys; dashed edges denote application-level ID references across database boundaries. Authentication uses users, roles, a join table, refresh tokens, and OTP records. Showtime storage relates rooms to seats and showtimes. Booking storage keeps ticket-seat price snapshots and concession lines. Payments relate to refunds within their own database [4], [7].

Cross-database references such as a payment's booking ID are not PostgreSQL foreign keys in the supplied schema. The reservation's booking ID is also not declared as an SQL foreign key in the inspected base script. These distinctions matter when discussing consistency: asynchronous handlers and application checks coordinate business records, while database constraints cover only their defined local scope. The figure is deliberately a partial ERD, not a claim to reproduce every table or every entity-mapping detail.

Initialization SQL and incremental migrations are kept in the database submodule. Compose mounts those files and runs an initialization script for an empty volume, followed by a migration job. Additional SQL under the backend tree updates movie metadata and reviews. The migration script excludes selected demo-seeding migrations. This inspected path uses SQL scripts; no claim is made that EF migration history governs the deployed schema [3], [4].

E. Payments and Event Delivery

The PayOS adapter creates checkout links, queries orders, and cancels links. It sends merchant headers and signs creation payloads with HMAC-SHA256. Webhook handling validates the data signature before using fields, matches the payment order and amount, and records the successful payment. A later booking-confirmation conflict can initiate a refund request. These operations are integrated in source; live merchant delivery was not exercised [7], [13].

Asynchronous delivery. Outbox publishers read unpublished events with `FOR UPDATE SKIP LOCKED`, publish persistent messages with publisher confirmations, and record publication state. Consumers handle booking cancellation, payment/refund changes, email notifications, and showtime cancellation. Event identifiers and uniqueness checks support duplicate handling. This follows an at-least-once delivery approach: a broker publish and database publication marker are not a single distributed transaction. RabbitMQ confirms are useful evidence of broker acceptance, not proof of exactly-once business effects or exactly-once email delivery [7], [14].

IV. WEB APPLICATION ARCHITECTURE AND IMPLEMENTATION

A. Entry Points and Browser Navigation

Both entry points restore a stored session before constructing their application. The customer router maps named paths to movie, showtime, booking, payment, notification, and account screens. Authentication-required routes redirect to login while retaining a pending destination. Detail routes accept an identifier and can reload their data when no in-memory navigation argument exists. This is particularly relevant to browser refresh and shared links [1].

The administration router classifies routes as public, staff, or admin. It checks roles before creating protected pages. This improves navigation but is not the security boundary: the backend independently authorizes management operations. CI defines a customer Web build and a separate admin build using `-base-href /admin/`. The configured build layout is evidence of packaging, not proof that a hosting environment serves both correctly [1], [15].

B. State and API Abstractions

`SessionState` extends `ChangeNotifier`; an `InheritedNotifier` exposes authentication, the current user, and unread notification count. Screens otherwise maintain local state for loading, selection, filters, and errors. No evidence requires labeling this state system as Bloc, Riverpod, or Redux. Repositories translate screen requests into service calls; the shared API client handles JSON, timeouts, bearer headers, and structured errors [1].

An authenticated 401 can trigger one refresh and a retry. Concurrent callers share a pending refresh future to avoid

TABLE III
MAJOR WEB FUNCTIONAL MODULES

Module	Implemented responsibility	Principal layers
Catalog	Browse, details, genres, trailers	Flutter; MovieService
Booking	Seats, summary, history, QR ticket	Flutter; BookingService
Checkout	PayOS link and status checks	Flutter; payment service
Counter	Walk-in booking and cash payment	Admin UI; booking/payment APIs
Management	Movies, rooms, schedules, users	Admin UI; authorized services
Retention	Promotions, points, reviews	Client views; booking database
Notifications	Lists, read state, email records	Flutter; event handlers

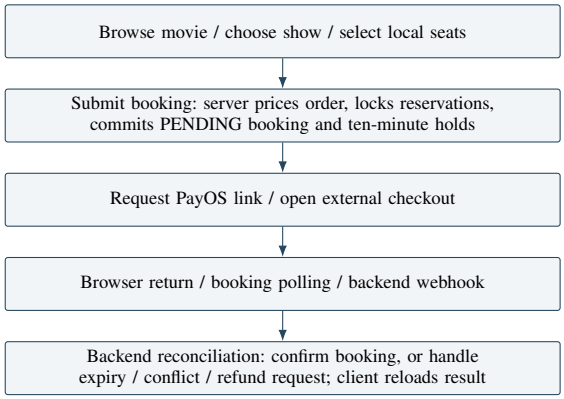


Fig. 3. Web reservation and checkout control flow. No client redirect or timer is itself proof of payment.

independently rotating the same token. Network failures retain the session where possible, whereas invalid refresh credentials or account-status errors clear it. `SessionApp` then replaces navigation with sign-in and displays the reason. These details connect the API layer to visible browser behavior instead of relying on route guards alone.

C. Responsive Presentation and Reuse

The layout utility switches the main desktop arrangement at 900 logical pixels. Wide customer pages use a top navigation frame; narrower layouts retain the application-style bottom navigation. Shared width constraints distinguish forms, reading views, and wide pages. `ResponsiveColumns` uses a separate local constraint threshold, defaulting to 820 pixels, to stack a sidebar below content. Consequently, there is not one universal breakpoint for every component [1], [2].

Theme files define colors, spacing, radii, and typography; common components provide buttons, panels, network-image fallbacks, loading, empty, and error states. Web-specific trailer implementations and counter-ticket printing use browser facilities through conditional imports. This is verified platform branching, not evidence that all browsers or assistive technologies were evaluated. Table III summarizes the main functional divisions.

V. CORE BUSINESS WORKFLOWS

A. Discovery to Reservation

The home screen requests now-showing or coming-soon data, genre filters, and title search. Search input is debounced by 350 milliseconds in source. Movie details retrieve open showtimes; the catalog repository bounds customer schedule selection to a seven-day window and can fetch subsequent pages while that window is relevant. Room seats and occupied seat IDs are requested in parallel and combined into a client map [1], [2].

Selection versus reservation. Selection is initially local. Every five seconds, the seat screen attempts to refresh availability while it is current; seats no longer free are removed from the local selection, including paired-seat adjustments. Submitting the summary creates the backend hold described in Section III. A conflict still requires the customer to choose again. The repository therefore implements polling plus transactional reservation, rather than a SignalR or WebSocket seat-lock channel [1], [7].

B. Pricing and Checkout

For ticket subtotal S, applicable promotion discount D, concession subtotal C, and redeemed loyalty points q, the booking code computes

$$T = S - D + C - 1000q. \tag{1}$$

The loyalty helper restricts redemption to at most $\lfloor 0.5(S - D + C)/1000 \rfloor$ points, additionally bounded by the customer's balance. Promotion calculation caps the discount at 90% of ticket value and rounds it down to a 1,000-VND increment. These are implementation constants, not measured economic outcomes [7].

Figure 3 summarizes checkout. The payment page opens the returned PayOS URL and displays the booking's expiry. It polls booking state every five seconds and invokes provider verification every sixth eligible poll. Browser return parameters identify an order to verify; the backend remains responsible for deciding payment success. A timer or a successful redirect alone does not confirm the ticket [1], [7].

C. Confirmation, Cancellation, and Operations

Confirmed bookings expose history/detail views and QR ticket content. Customers can request cancellation subject to server policy; the default paid-booking cutoff is two hours before the show. A customer refund request sets `REFUND_PENDING` and retains seats until the refund decision. Staff complete or reject refund processing through privileged APIs. The code records manual transfer references; it does not implement an automatic banking refund through a PayOS refund endpoint [2], [7].

Staff workflows include counter booking, cash collection, booking lookup, check-in, showtime operations, payment records, and notification handling. Admin screens additionally

manage movies, rooms, users, concessions, promotions, and membership data. These are source-backed screen-to-service paths, not a claim that every management path was manually exercised. Check-in requires an eligible paid booking and a server-configured time window; repeated admission is rejected by business logic [2], [7].

VI. Security and Data Protection

A. Trust Boundaries

The implementation places checks at three trust boundaries [5], [7], [10]:

- **Identity and ownership.** Gateway/service authorization accompanies UI routing. Booking reads check ownership; self-service creation uses identity claims.
- **Order value.** Server pricing recalculates seat and concession amounts instead of trusting totals submitted by the browser.
- **Provider data.** Webhook signatures are checked before payment data is used to confirm a booking.

Password validation additionally accounts for BCrypt's input-length boundary.

B. Browser Storage and Configuration

The client actively uses `flutter_secure_storage` for access/refresh tokens and user metadata. Writes are sequential, with a source comment explaining first-write key creation on Web. Unreadable storage clears local credentials. This establishes the chosen storage abstraction, not an application-level guarantee against script execution or browser compromise. The package documents HTTPS or localhost requirements for its Web backend [1], [16].

Deployment configuration. Gateway CORS uses configured origins when present, but falls back to allowing any origin if the list is empty. Fixed-window IP rate limits include a separate authentication policy and return 429 responses. Compose passes secrets via environment variables; the review did not open or publish credential values. The supplied local topology exposes HTTP, so TLS termination is a deployment requirement to verify, not an observed property of a live website. Seed accounts and demonstration data should not be treated as a production identity baseline [3]–[5].

VII. Testing, Verification, and Evaluation

A. Method and Observed Results

Verification was performed on October 28, 2026. Table IV separates successful checks from environmental blocks and external operations that were not attempted. No response-time, load, availability, coverage, or usability metrics were collected. Build success demonstrates compilation, while unit and widget results support only the exercised assertions [15], [17].

After restoring the existing declared Flutter dependencies, the full `flutter test -no-pub -reporter expanded` run passed 209 tests. `flutter analyze -no-pub` reported one informational finding, `curly_braces_in_flow_control_structures`, in `trailer_url.dart:12`; no analysis errors or warnings remained. The analyzer was therefore not a clean zero-finding run. The earlier no-restore attempt failed because locally cached QR/sharing packages were missing, and is not counted as a passed test run. No application source was changed to obtain the restored-package result.

TABLE IV
Verification Executed on October 28, 2026

Check and method	Observed result / status
BookingService isolated unit filter	**PASS**: 17 tests, zero failed
ApiGateway test project	**PASS**: 21 tests, zero failed
AuthService build	**PASS**: zero errors/warnings
MovieService build	**PASS**: zero errors; four nullable warnings
Flutter analysis	1 informational lint; no errors/warnings
Flutter unit/widget suite	**PASS**: 209 tests after package restore
PostgreSQL/RabbitMQ integration suites	**BLOCKED**: Docker daemon unavailable
SQL/migration execution	**NOT EXECUTED**: no running database
Live PayOS, SMTP, deployed acceptance	**NOT EXECUTED**: external services not exercised
Customer/admin browser builds	**PASS**: both entry points compile

Both browser entry points also compiled successfully with `flutter build web -no-pub -no-web-resources-cdn`; the admin build selected `lib/main_admin.dart` and base path `/admin/`. These are compilation results, not browser-hosted acceptance tests.

The 17 isolated BookingService tests exercise signature handling and email templates. The 21 gateway tests exercise request policy and failure behavior with test infrastructure. Neither total should be interpreted as a live deployment acceptance count. AuthService compiled without warnings; MovieService compiled with four nullable-analysis warnings. BookingService and ApiGateway compiled as part of their successful test runs.

B. Test Doubles and Missing Runtime Evidence

Booking integration fixtures use PostgreSQL Testcontainers and optionally a real RabbitMQ container, but replace MovieService, PayOS, and SMTP with fake clients. Flutter workflow tests likewise use fake repositories and plugin substitutes. Their test names describe intended assertions, not evidence of real merchant payments or actual email delivery [15].

Runtime verification boundary. Docker's daemon was unavailable during this review, preventing the container-dependent suites and a complete Compose deployment from being verified. SQL files and EF mappings were inspected directly, but no live-schema or migration execution result is claimed. Browser-hosted checkout, external trailer delivery, printing, and public HTTPS hosting were not manually validated. These limitations are retained even when a source path and a unit test exist.

VIII. Limitations and Future Development

A. Current Limitations

The two browser applications share a Flutter project, so reports of independent Web and mobile products should not imply independent source ownership. Runtime availability depends on gateway, identity, data, and messaging services; the account-status call adds a dependency on AuthService for protected requests. Polling provides periodic updates rather than immediate server push. Temporary refresh failures deliberately retain older availability, making the booking transaction the final arbiter [1], [5], [7].

Refund transfer is a staff operation. Notification handling records failed email sends for staff retry; database deduplication does not make SMTP and database persistence atomic. Local-time timestamps and legacy Npgsql behavior are explicitly used, limiting claims about multi-time-zone deployment. The base SQL includes demonstration payment method labels such as VNPAY and MOMO; no active integrations for those providers were established by the inspected payment adapter. Missing runtime evidence and compiler warnings further limit claims of operational readiness [3], [4], [7].

B. Recommendations

Future work should first execute the container suites and migrations on a controlled environment, add live provider acceptance checks with safe merchant test arrangements, and verify browser reloads and payment returns under HTTPS. Recommendations also include resolving nullable warnings, validating CORS and secret settings for a named deployment, and testing keyboard/screen-reader operation and supported browser printing. A push-based availability channel could be evaluated if polling becomes insufficient; it is not a committed roadmap item or a feature of the inspected version.

IX. Conclusion

CosmoQ's Web implementation combines Flutter customer and administration entry points with a shared ASP.NET Core service backend. Its most substantive mechanisms are source-backed pricing and ownership checks, transactional seat holds, payment verification, and event-mediated notification/refund processing. SQL and entity mappings clarify both local constraints and cross-database consistency responsibilities. Successful isolated backend checks substantiate specific implementation properties, while the reported runtime limitations prevent broader claims about production security, live payments, or capacity. The resulting system is a documented engineering implementation whose next evaluation step is controlled end-to-end deployment verification.

Acknowledgment

The student developers thank Mr. Huynh Vang, Department of Software Engineering, FPT University Quy Nhon Campus, for his guidance as project advisor and for supporting the team's software engineering work on CosmoQ Cinema.

References

[1] CosmoQ Cinema Student Developers, "Flutter client entry points, routing, session state, and API adapters," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_Mobile/lib, inspected Oct. 28, 2026.

[2] ——, "Customer browser layout and administration workflows," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_Mobile/lib/features, inspected Oct. 28, 2026.

[3] ——, "Compose deployment and database initialization scripts," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE, inspected Oct. 28, 2026.

[4] ——, "Cinema database SQL schemas and migrations," Repository source, revision e455d297c76b, https://github.com/PRM393-Cinema/Project-Cinema-DB/tree/e455d297c76b14e3fd6aa8c0af4bb11f77ec6a2b, inspected Oct. 28, 2026.

[5] ——, "Gateway policies, reverse proxy, and account-status middleware," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE/ApiGateway/ApiGateway, inspected Oct. 28, 2026.

[6] ——, "Movie, room, showtime, and seat-pricing implementation," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE/MovieService/MovieService, inspected Oct. 28, 2026.

[7] ——, "Booking, payment, refund, loyalty, notification, and messaging implementation," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE/BookingService/BookingService, inspected Oct. 28, 2026.

[8] Flutter, "Architecting Flutter apps," https://docs.flutter.dev/app-architecture, accessed Oct. 28, 2026.

[9] Npgsql, "Npgsql Entity Framework Core provider," https://www.npgsql.org/efcore/, accessed Oct. 28, 2026.

[10] CosmoQ Cinema Student Developers, "Authentication, account validation, JWT generation, and user management," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/Cinema_BE/AuthService/AuthService, inspected Oct. 28, 2026.

[11] Microsoft, "Configure JWT bearer authentication in ASP.NET Core," https://learn.microsoft.com/en-us/aspnet/core/security/authentication/configure-jwt-bearer-authentication, accessed Oct. 28, 2026.

[12] PostgreSQL Global Development Group, "PostgreSQL 17: Constraints," https://www.postgresql.org/docs/17/ddl-constraints.html, accessed Oct. 28, 2026.

[13] payOS, "payOS API," https://payos.vn/docs/api/, accessed Oct. 28, 2026.

[14] RabbitMQ, "Consumer acknowledgements and publisher confirms," https://www.rabbitmq.com/docs/confirms, accessed Oct. 28, 2026.

[15] CosmoQ Cinema Student Developers, "Backend tests and Flutter verification workflows," Repository source, revision 55642353331f, https://github.com/PRM393-Cinema/Cinema/tree/55642353331f7119c16d4fef05f54bfc559cae8b/., inspected Oct. 28, 2026.

[16] flutter_secure_storage Maintainers, "flutter_secure_storage 11.2.0," https://pub.dev/packages/flutter_secure_storage, accessed Oct. 28, 2026.

[17] Flutter, "Testing Flutter apps," https://docs.flutter.dev/testing/overview, accessed Oct. 28, 2026.